

Mintzy Desktop App — Complete Plan & Required Changes

Prepared for team review — July 16, 2026

Table of Contents

1. Overview
2. Desktop App Architecture
3. Complete Auth Flow
4. Required Backend Changes
5. Required Frontend Changes
6. Desktop App Implementation (Current Status)
7. Data Flow Diagrams
8. API Contracts
9. Timeline

1. Overview

Product: Native Windows desktop wrapper for the Mintzy Plugin terminal.

Scope: Plugin product only. The app loads the real Mintzy website in a sandboxed Electron BrowserWindow.

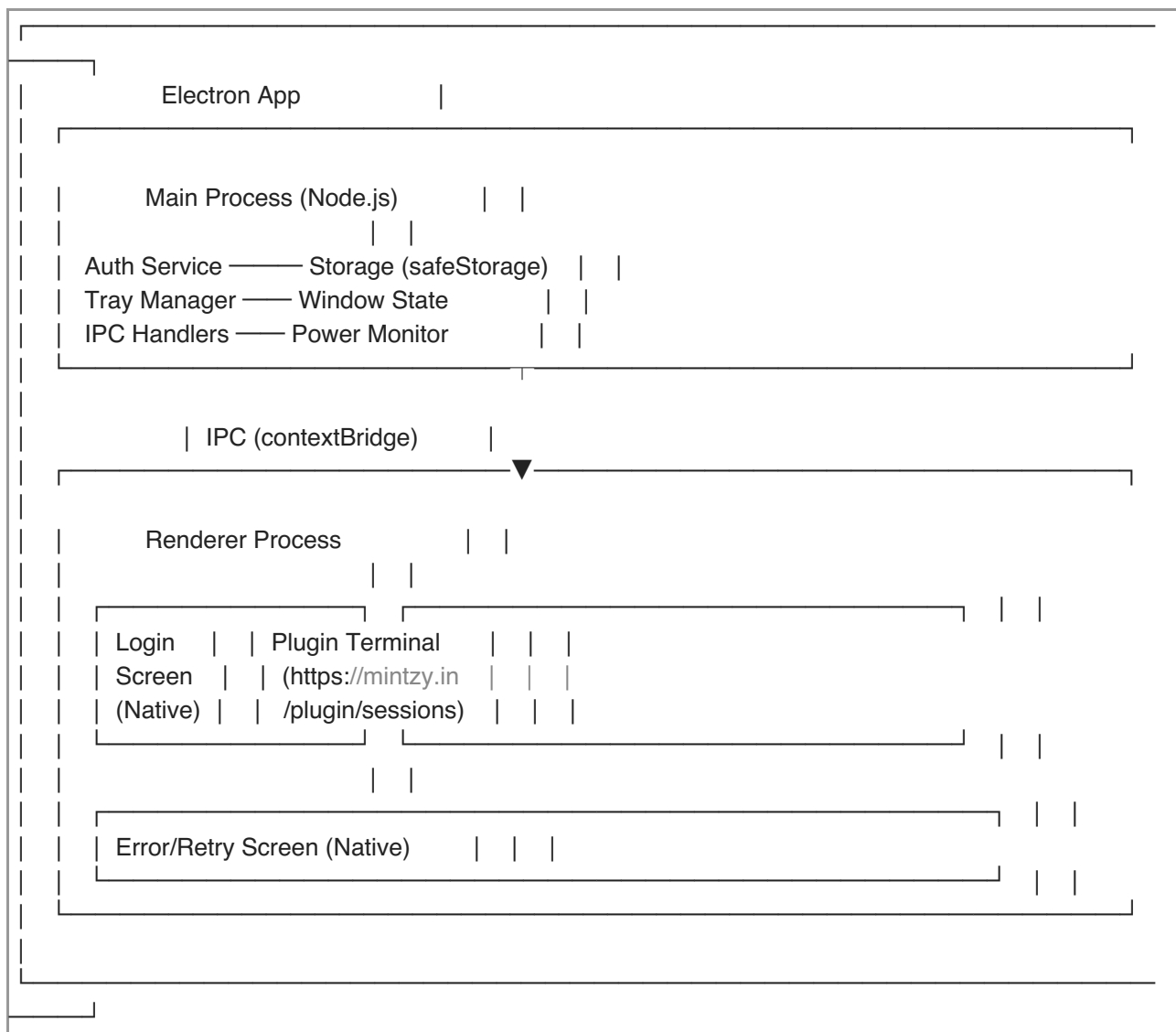
Auth: API-key based (sk_trade_*). No Google OAuth in the desktop app.

Broker type: Determined at API key generation time, embedded in the key's metadata, auto-applied by the desktop app.

Why a Desktop App?

- Persistent terminal session (not dependent on browser tab)
- System tray integration (minimize, notifications, auto-launch)
- Single-instance lock (prevent duplicate trading sessions)
- Encrypted credential storage (Windows DPAPI via Electron safeStorage)
- Sleep/resume handling (reconnect websockets after laptop sleep)
- Offline/error screens (instead of blank browser pages)

2. Desktop App Architecture



Security Hardening

Setting	Value	Why
contextIsolation	true	Renderer can't access Node.js
nodeIntegration	false	No require() in renderer
sandbox	true	Limits blast radius if web content is compromised
preload	Minimal contextBridge	Only exposes auth/navigation/system APIs
DevTools	Production disabled	No console access for users

Credential Storage

- API key encrypted at rest using safeStorage (Windows DPAPI)
- Stored at `app.getPath('userData')/credentials.enc`
- Never plaintext, never in logs or console output
- Logout clears encrypted file
- Uninstall via NSIS `deleteAppDataOnUninstall: true` cleans up

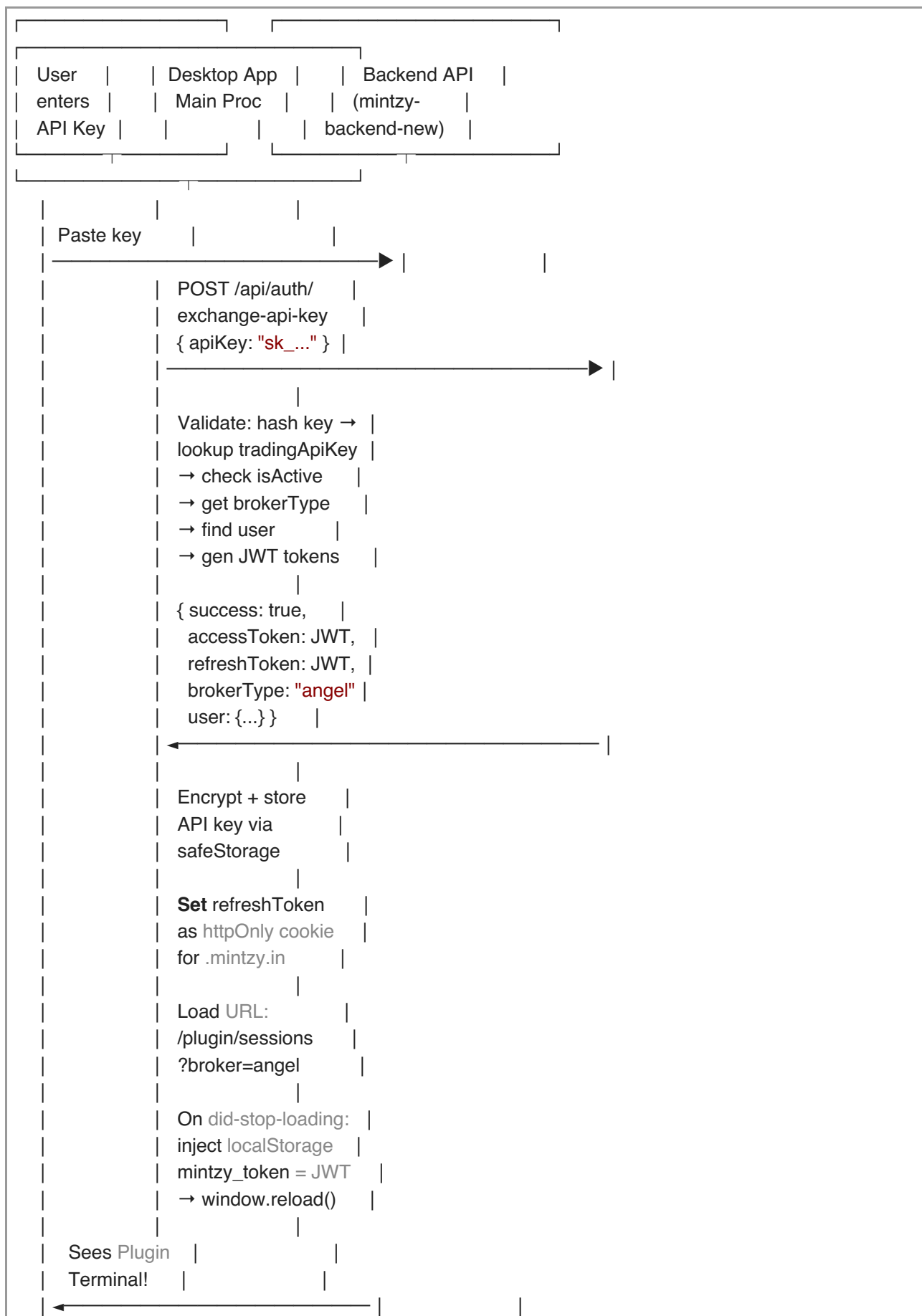
3. Complete Auth Flow

3.1 API Key Generation (on Website)

User logs into website (Google OAuth)

- Navigates to Account Settings → API Keys
- Clicks "**Generate New Key**"
- Enters: Name, selects Broker **Type** (Angel One / TradeX)
- Backend creates key with brokerType stored
- **User copies** the generated sk_trade_... key

3.2 Desktop App Login



3.3 Relaunch (Already Logged In)

App starts

- **Check** stored API key **in** safeStorage
- **Call** exchange endpoint **with** stored key
- **If valid**: **get** fresh tokens, **load** terminal
- **If invalid/expired/revoked**: **show login** screen

3.4 Token Refresh Mid-Session (Automatic)

Access token expires (7 min)

- Website's axios interceptor catches 401
- Calls POST /api/auth/refresh
- httpOnly refresh cookie **is** sent automatically
- Backend issues **new access** token
- Interceptor retries original request
- User never notices

If refresh token also expired (7 days):

- Desktop app detects → shows login screen
- User re-enters API key → fresh tokens

4. Required Backend Changes

4.1 Add brokerType to tradingApiKey Model

File: models/tradingApiKey.js

```
// New field:
brokerType: {
  type: String,
  enum: ['angel', 'tradex'],
  required: true,
}
```

4.2 Update Key Generation Endpoint

File: Routes/tradingApiKey.routes.js / Controllers/tradingApiKeyController.js

Current: POST /api/plugin-keys/generate accepts { name }

Change: Accept { name, brokerType }, validate brokerType, store with key

4.3 New: API Key Exchange Endpoint

File: Routes/auth.js + auth controller/service

POST /api/auth/exchange-api-key

Auth: None (key is in body)

Body: { apiKey: "sk_trade_..." }

Validation:

1. SHA-256 hash the apiKey
2. Lookup in tradingApiKey collection
3. Fail if not found or isActive === false
4. Get brokerType from the key document
5. Find user by userId on the key
6. Generate accessToken (7m expiry) via generateAccessToken()
7. Generate refreshToken (7d expiry) via generateRefreshToken()
8. Return { success, accessToken, refreshToken, brokerType, user }

Error responses:

```
{ success: false, message: "Invalid API key" } // key not found
{ success: false, message: "API key has been revoked" } // isActive === false
{ success: false, message: "User not found or inactive" } // user issue
```

4.4 No Changes Needed

- Plugin trading endpoints (buy/sell/etc.) — broker type is already sent with credentials
- authMiddleware — already handles JWT tokens correctly
- pluginAuthMiddleware — already handles both JWT and API key Bearer tokens

5. Required Frontend Changes

5.1 API Key Management Page (Account Settings)

New page: /settings/api-keys

Authentication: Google OAuth (existing)

Functionality:

- List existing keys (show prefix, name, brokerType, created date, last used)
- "Generate New Key" button → modal/form with:
 - Name (text input)
 - Broker Type (Angel One / TradeX dropdown)
- Show generated key once (like GitHub GPG key flow)
- Copy-to-clipboard button
- Revoke button per key

5.2 ConnectBrokerForm — Accept initialBrokerType

File: src/components/plugin/ConnectBrokerForm.tsx

```
interface Props {
  onSuccess: (sessionId: string, requiresTotp: boolean, brokerType: BrokerType) => void;
  onBack: () => void;
  initialBrokerType?: BrokerType | null; // NEW
}
```

Changes:

- If initialBrokerType is set:
 - Initialize brokerType state to it
 - Hide the broker dropdown entirely
 - Show a read-only label: "Broker: Angel One"
- User still enters credentials (API key, client code, password)
- Submit sends broker_type: initialBrokerType as before

5.3 PluginSessionPage — Read ?broker= from URL

File: src/app/plugin/sessions/page.tsx

```
// On mount, read query param
const [brokerFromUrl] = useState<BrokerType | null>(() => {
  if (typeof window !== 'undefined') {
    const params = new URLSearchParams(window.location.search);
    const b = params.get('broker');
    if (b === 'angel' || b === 'tradex') return b;
  }
  return null;
});

// Pass to ConnectBrokerForm
{currentView === 'broker' && (
  <ConnectBrokerForm
    initialBrokerType={brokerFromUrl}
    onSuccess={handleBrokerSuccess}
    onBack={() => setCurrentView('empty')}
  />
)}
```

6. Desktop App Implementation Status

Current State (Phase 1.1 & 1.2 Complete)

Branch: dev-anubhav on github.com/Mintzy-V1/Plugin-App-Desktop

Component	Status	Details
Electron scaffold	Done	Electron v43, contextIsolation, sandbox
electron-builder	Done	NSIS, per-user, Win x64, unsigned
Single-instance lock	Done	Second launch focuses existing window
Window state persistence	Done	Remembers size/position across restarts

safeStorage encryption	Done	DPAPI for credentials
Login screen (native)	Done	API key input with error states
Error/retry screen	Done	Network vs broker expiry distinction
Mocked auth service	Done	Test keys: valid/expired/broker-expired
IPC wiring	Done	Login → auth → terminal flow
Token injection mechanism	Done	localStorage injection via executeJavaScript
Plugin URL	Configured https://www.mintzy.in/plugin/sessions	
Broker URL param	Phase 1.3 Append ?broker= to URL	

Remaining Work

Phase 1.3 — Terminal Loading Spike

- [] Append ?broker= query param when loading Plugin URL
- [] Set refresh token as httpOnly cookie via session.cookies.set()
- [] Test against real Plugin URL (currently blocked on exchange endpoint)

Phase 2 — Core Wrapper

- [] Silent revalidation on relaunch (exchange stored API key for fresh tokens)
- [] Handle token refresh failures (API key revoked → login screen)
- [] Offline/error retry screen (already built, may need refinement)

Phase 3 — Native Features

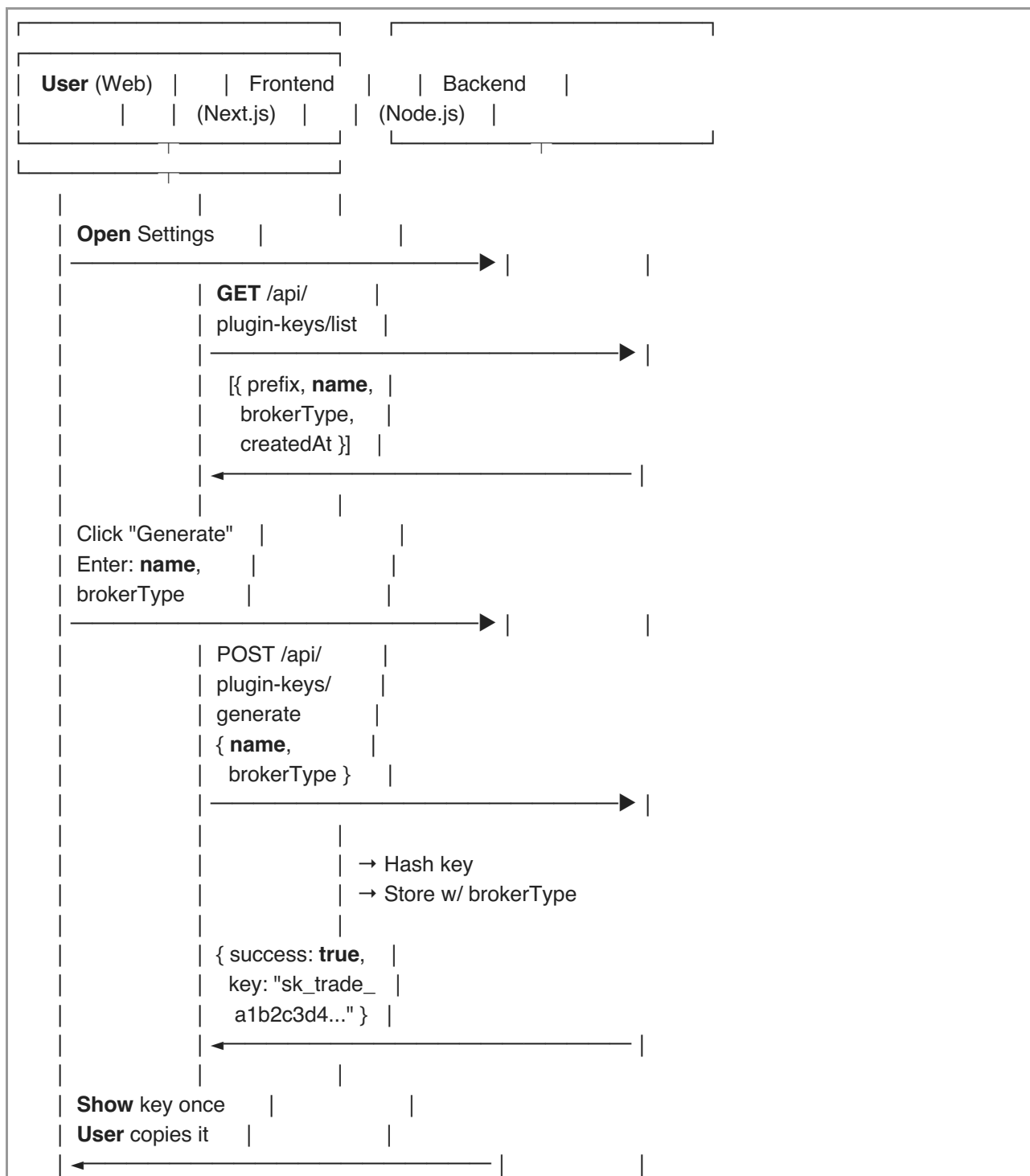
- [] Tray icon + minimize-to-tray
- [] Tray menu: Open, Logout, Quit
- [] Auto-launch toggle (Windows startup)
- [] Sleep/resume reconnect handling
- [] Notifications (if backend websocket available)

Phase 4 — Real Auth Integration

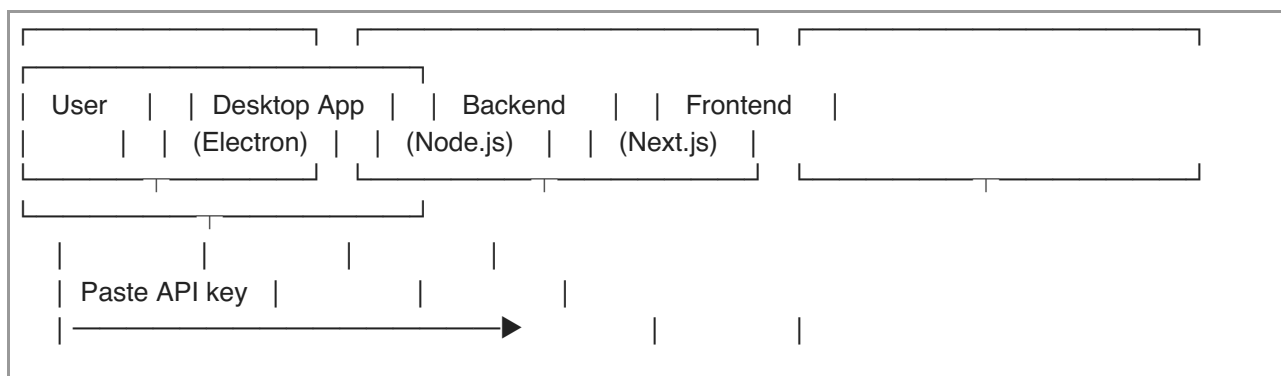
- [] Replace mocked auth with real exchange endpoint
- [] Build installer with Mintzy branding
- [] Full integration test with real accounts

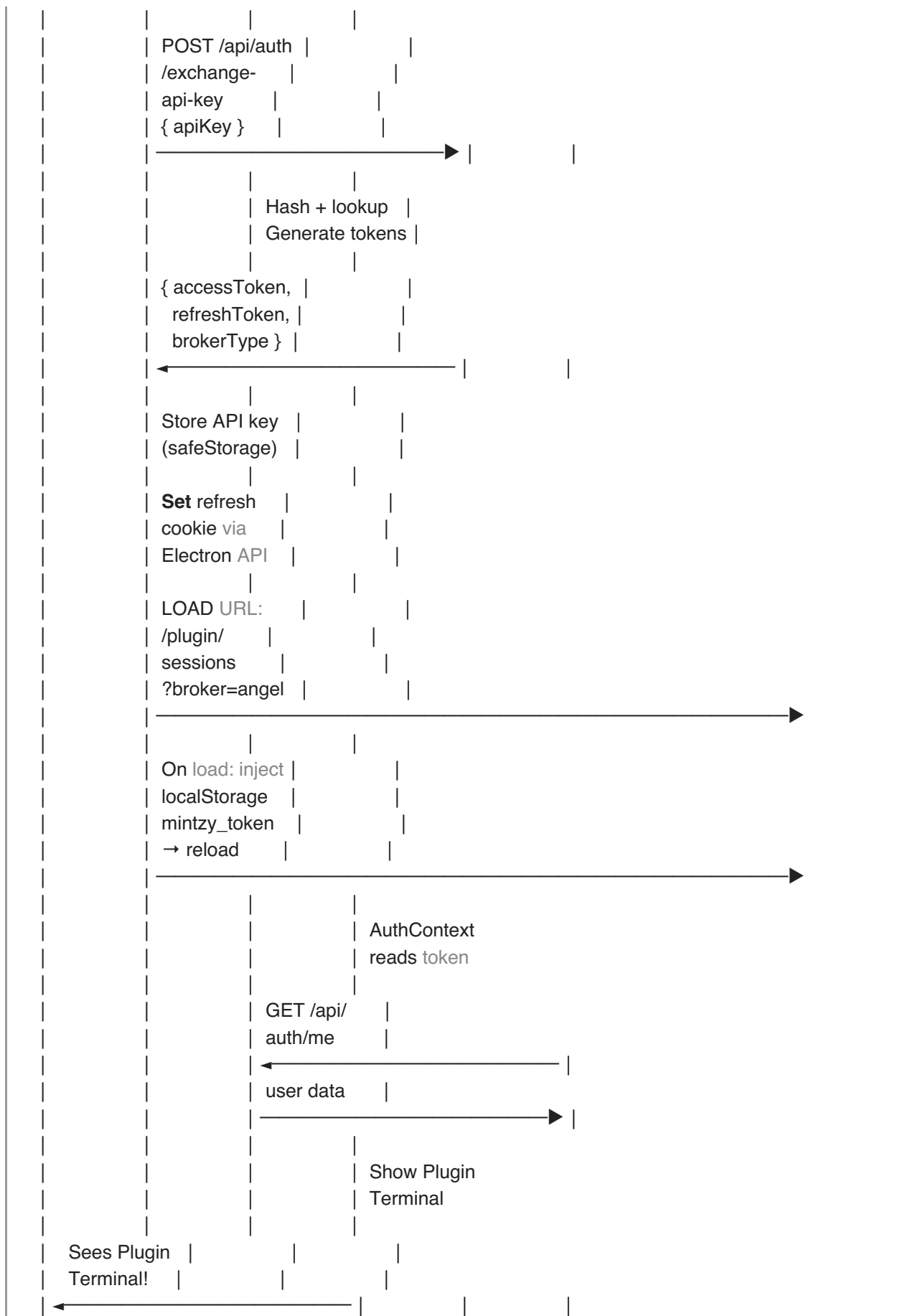
7. Data Flow Diagrams

7.1 API Key Generation

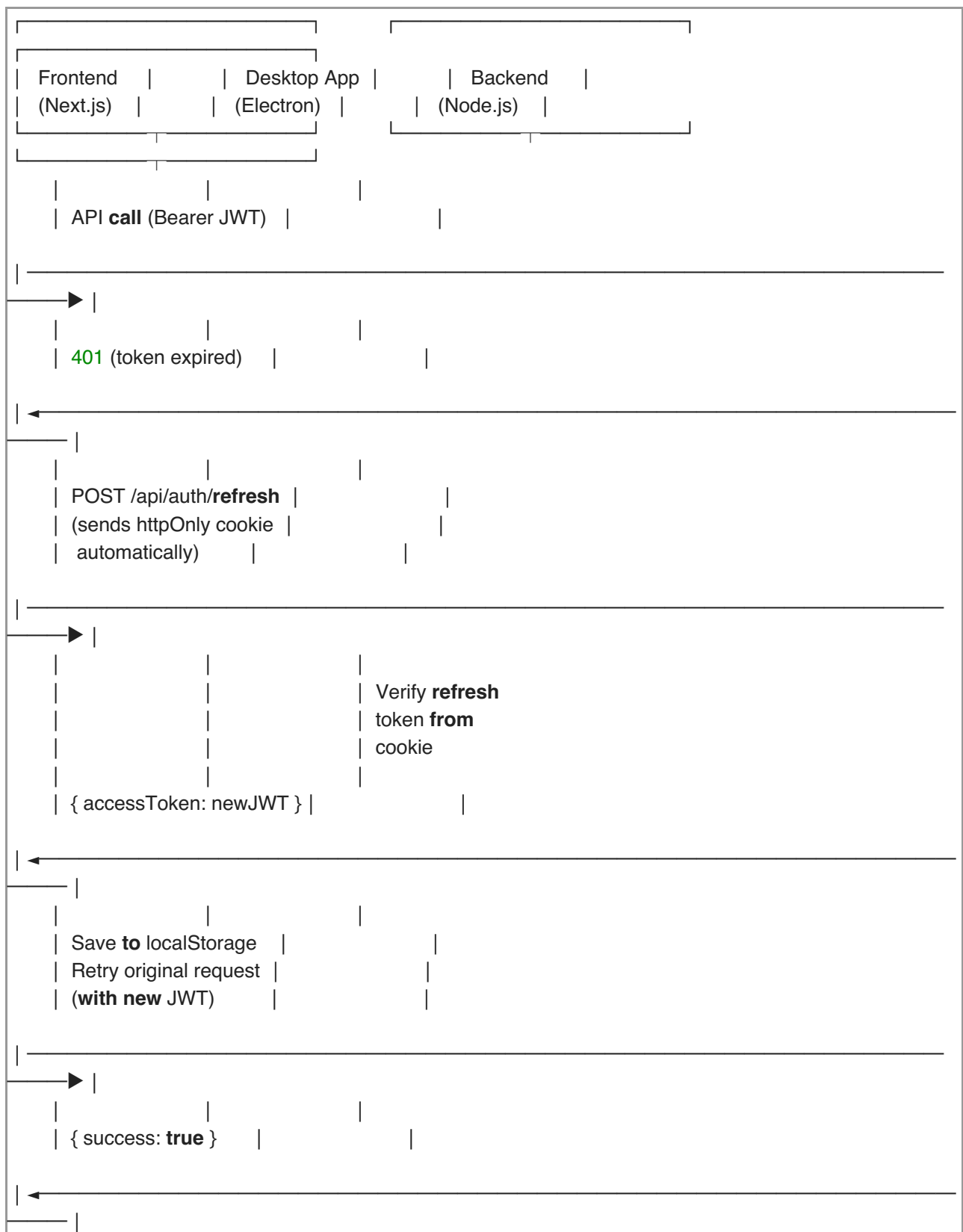


7.2 Desktop Login (Complete Flow)





7.3 Token Refresh (Automatic, Mid-Session)



Key: The refresh cookie was set by the desktop app via `session.cookies.set()` on initial login. The website's axios interceptor handles everything automatically after that.

8. API Contracts

8.1 Exchange API Key

POST `/api/auth/exchange-api-key`

Content-Type: application/json

Request:

```
{
  "apiKey": "sk_trade_a1b2c3d4e5f6..."
}
```

Response (200 - Success):

```
{
  "success": true,
  "accessToken": "eyJhbGciOiJIUzI1NiIs... ", // JWT, 7m expiry
  "refreshToken": "eyJhbGciOiJIUzI1NiIs... ", // JWT, 7d expiry
  "brokerType": "angel", // "angel" | "tradex"
  "user": {
    "_id": "64a1b2c3d4e5f6...",
    "name": "User Name",
    "email": "user@example.com"
  }
}
```

Response (401 - Invalid/Revoked):

```
{
  "success": false,
  "message": "Invalid API key" // or "API key has been revoked"
}
```

Response (404 - User not found):

```
{
  "success": false,
  "message": "User not found or inactive"
}
```

8.2 Generate API Key (Updated)

```

POST /api/plugin-keys/generate
Authorization: Bearer <JWT-from-OAuth>
Content-Type: application/json

Request:
{
  "name": "My Desktop Key",
  "brokerType": "angel"      // NEW: "angel" | "tradex"
}

Response (201):
{
  "success": true,
  "key": "sk_trade_a1b2c3d4e5f6...", // shown only once
  "keyId": "64a1b2c3d4e5f6..."
}

```

8.3 Plugin Terminal URL (Updated)

```

URL: https://www.mintzy.in/plugin/sessions?broker=angel
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
      NEW: optional query param
      Values: "angel" | "tradex"

If present: ConnectBrokerForm hides broker dropdown
If absent: ConnectBrokerForm shows broker dropdown (normal OAuth flow)

```

9. Timeline

Phase	Days	What
Backend changes	2-3 days	tradingApiKey model update + exchange endpoint
Frontend changes	3-4 days	API keys UI + ConnectBrokerForm broker param
Desktop Phase 1.3	0.5 day	Append ?broker= param, cookie setting
Desktop Phase 2	3 days	Session persistence, error handling, offline screen
Desktop Phase 3	3 days	Tray, auto-launch, sleep/resume, notifications
Desktop Phase 4	3 days	Real auth swap, build, buffer
Total	~2.5 weeks	

Dependencies:

- Desktop Phase 4 blocked on backend exchange endpoint
- Desktop end-to-end testing blocked on both backend + frontend changes
- Frontend API Keys page can be built independently of desktop app

End of document